전자회로실험 · STM32 실습

STM32_1
인터럽트와 PWM

결과보고서

실험일 2026 년 10 월 19 일

작성자 주윤찬

학번 2023440132

작성일 2026 년 10 월 26 일

과제 3·4 / FPGA 피아노·신호등·ADC

GitHub: YunchanJu/lab4_stm32_0

제출 태그: stm32-1-results-20261019-v2

1. 실험 목적과 구현 방법

1.1 목적 및 실험 구성

10 월 19 일 실습은 버튼 외부 인터럽트, TIM6 주기 인터럽트와 TIM3 PWM 을 이용해 폴링 방식의 LED 제어를 확장하는 실험이다. 과제 3 은 버튼 입력으로 LED 상태 변경 간격을 선택하고, 과제 4 는 PWM 듀티를 바꾸며 C 디버거와 오실로스코프에서 이를 확인한다.[1]

FPGA 개별 실습에서는 피아노·신호등·외부 ADC 를 다루었다. 키 입력에 따른 음 출력, 신호등의 상태 전이와 점멸, ADC 의 직렬 데이터 수신을 시뮬레이션과 보드 시연으로 확인하였다.[1,2]

항목	구성 및 관측
MCU	NUCLEO-G474RE, STM32 HAL
버튼 / LED	외부 인터럽트 / 내장 LED PA5
타이머	TIM6 100ms 인터럽트
PWM	TIM3 CH1, PB4-GND 계측
개발 / 계측	VS Code 디버거, Vivado, Tektronix TBS2000B

1.2 회로와 신호 경로

버튼 입력 → EXTI 콜백 → button_isr_p7()로 처리한다. TIM6 업데이트 → HAL_TIM_PeriodElapsedCallback() → tim6_isr_p4()에서 누적 횟수를 세어 PA5 를 토글한다. 두 경로의 상태를 공유하되 메인 루프에서는 버튼·LED 시간을 다시 처리하지 않는다.

PWM 은 PB4 출력과 계측기 CH1, 공통 GND 를 연결한다. 내장 LED 의 PA5 와 PWM 출력 PB4 는 서로 다른 핀이므로 LED 밝기를 PWM 파형의 대리 지표로 삼지 않았다. 보드 구성은 ST 의 UM2505 를 참고하였다.[3]

1.3 타이머와 PWM 계산

MCU 설정의 타이머 입력을 100MHz 로 두면 업데이트 주기는 (PSC+1)(ARR+1)/fTIM 이다. TIM6 의 PSC=9999, ARR=999 에서 0.1 초가 되고, 10 번 누적하면 LED 상태 변경 간격 1 초가 된다.

구분	계산값
TIM6	100MHz /10000 /1000 =10Hz
TIM3 PWM	100MHz /100 /1000 =1kHz
PWM 한 주기	1ms
듀티	CCR /(ARR+1) ×100%

TIM3 의 PSC=99, ARR=999 에서 CCR=250·500·750 은 각각 25·50·75%를 뜻한다. CCR 을 바꾸는 것은 비교 시점을 바꾸는 것이며 주기 250·500·750 으로 변경하는 것이 아니다.

1.4 실험 확인 항목

MCU 에서는 버튼 입력에 따른 LED 간격 변경과 PWM 듀티 순환을 확인하였다. 과제 3 은 버튼과 TIM6 인터럽트의 역할을 분석하고, 과제 4 는 Watch 의 CCR 값과 오실로스코프의 듀티 측정값을 비교하였다.

FPGA 에서는 키 입력과 음 출력, 신호등 출력 비트의 순서, ADC 의 채널 선택과 수신 완료 신호를 확인하였다. 테스트벤치는 정상 입력뿐 아니라 무입력, 동시 입력, 모드 전환 및 리셋 조건을 포함한다.

전체 실험 소스

2. 과제 3 · 인터럽트 기반 LED 간격 변경

2.1 ISR 역할 분리

버튼 ISR 은 200ms 이내의 반복 입력을 제거하고 간격을 10→20→40→80→10 tick 으로 변경한다. TIM6 ISR 은 100ms 마다 카운터를 증가시켜 선택된 값에 도달했을 때 LED 를 토글한다. 이때의 1·2·4·8 초는 LED 가 켜짐 또는 꺼짐 상태에서 바뀌는 간격이며 전체 점멸 주기는 그 두 배이다.

```
/* core logic, abbreviated */
interval_ticks *= 2U;
if (interval_ticks > 80U)
    interval_ticks = 10U;

if (++count >= interval_ticks) {
    count = 0U;
    HAL_GPIO_TogglePin(PORT, PIN);
}
```

공유 상태는 volatile 로 선언한다. volatile 은 인터럽트 간 재읽기를 보장하는 용도이며 복합 연산의 원자성을 대신하지 않는다. 버튼 처리 코드에서는 버튼 ISR 의 간격 변경과 카운터 초기화 구간을 PRIMASK 저장·복원으로 보호한다. TIM6 우선순위 4, 버튼 7 조건을 고려했다.[1]

선택값	상태 변경 간격	전체 점멸 주기
10 tick	1 초	2 초
20 tick	2 초	4 초
40 tick	4 초	8 초
80 tick	8 초	16 초

polling_routine()은 비워 두고 ISR 안에서 printf, 입력 대기, HAL_Delay 를 사용하지 않았다. 간격 변경 직후 첫 토글은 TIM6 의 현재 위상에 따라 최대 100ms 일찍 나타날 수 있다.

과제 3 소스·실제 영상

2.2 보드 시연 주요 장면



그림 1. 버튼 인터럽트 기반 LED 제어의 보드 구성.

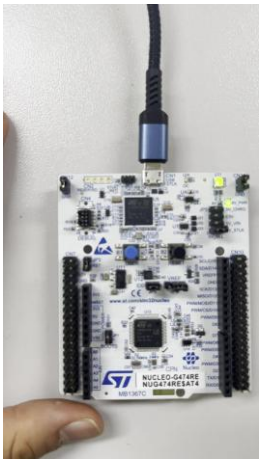


그림 2. 과제 3 LED 점멸 동작.

2.3 예상과 관측의 비교

LED 간격은 TIM6 의 100ms 기준 신호를 10·20·40·80 회 누적하여 설정한다. 버튼 ISR 에서 선택값을 바꾸고 타이머 ISR 에서 LED 를 갱신하므로, 메인 루프에서 시간을 계속 검사하지 않아도 동작을 유지할 수 있다.

과제 3 을 포함한 다섯 동작 모드의 ARM 빌드가 정상적으로 완료되었다. LED 간격의 이론값은 1·2·4·8 초이며, 시간 정확도는 PA5 의 연속 토글 간격을 계측해 평가한다. 버튼 입력 직후에는 TIM6 위상에 따른 차이가 발생할 수 있다.

3. 과제 4 · PWM 변경 및 측정 결과

3.1 디버거에서 상태 순환 확인

버튼 입력마다 g_pwm_step 을 순환시키고 CCR=250×(step+1)을 기록한다. TIM3 CH1 비교값을 갱신하며 주파수 설정은 유지한다. Watch 에서 확인한 순서는 250→500→750→250 이다.

```
g_pwm_step=(g_pwm_step+1U)%3U;
g_pwm_ccr=250U*(g_pwm_step+1U);
__HAL_TIM_SET_COMPARE(&htim3,
    TIM_CHANNEL_1,g_pwm_ccr);
```

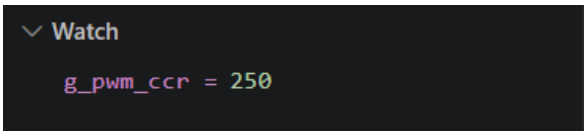


그림 3. Watch 에서 확인한 초기 CCR 값 250.



그림 4. 버튼 처리 후 CCR 500.

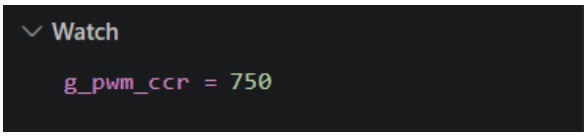


그림 5. 다음 처리 후 CCR 750.

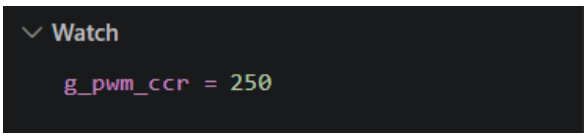


그림 6. 다시 CCR 250 으로 복귀.

중단점에서 멈춘 시간은 정상 실행 시간 측정에 포함할 수 없다. Watch·F10 은 상태 갱신 순서 확인에 사용하고, 파형 측정은 중단점을 해제하여 실행한 계측 화면을 근거로 삼는다.[1]

설정	실측 듀티	오차(%p)
25%	24.79%	-0.21
50%	50.34%	+0.34
75%	74.98%	-0.02

오차는 실측값-설정값의 퍼센트포인트 차이다. 상대 오차율과 구별한다.

3.2 오실로스코프 실측

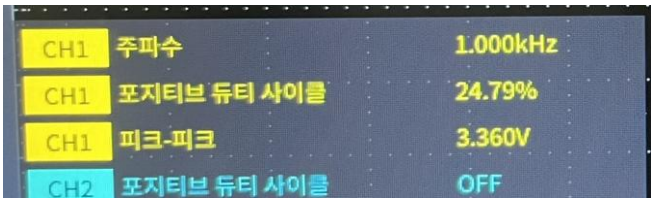


그림 7. CCR 250: 1.000kHz, 24.79%, 3.360Vpp.



그림 8. CCR 500: 1.000kHz, 50.34%, 3.360Vpp.



그림 9. CCR 750: 1.008kHz, 74.98%, 3.360Vpp.

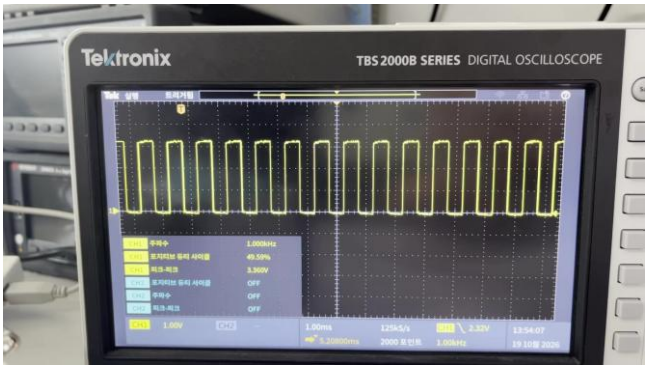


그림 10. 버튼 입력에 따른 PWM 출력 파형.

측정 듀티는 설정값에 가깝고 최대 편차는 0.34%p 이다. 주파수는 각각 1.000·1.000·1.008kHz 였다. 마지막 측정의 편차는 목표 대비 +0.8%이며, 원인을 분석하려면 계측 조건을 일정하게 유지한 반복 측정과 타이머 기준 클록 확인이 필요하다.

측정 주파수와 듀티로 계산한 HIGH 시간은 약 247.90·503.40·743.85µs 이다. 이 값은 HIGH 시간=듀티/주파수 관계로 산출하였다. 세 조건 모두 피크 대 피크 전압은 3.360V 였다.

PWM 파형 및 측정값

4. FPGA 추가 1·2 · 피아노와 신호등

4.1 피아노: 입력별 음 출력

피아노는 키 입력에 따라 주파수를 선택하고 반주기마다 piezo 출력을 반전하는 구조이다. 시뮬레이션에서는 8 개 음과 무입력, 복수 키, 리셋 조건을 검사하였으며, 1476ns 에서 PASS 를 확인하였다. 실제 음높이는 보드 클록과 분주값으로 결정된다.[2]

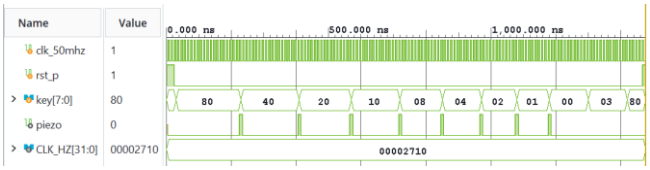


그림 11. 키 입력에 따른 piezo 출력 시뮬레이션.

```
PASS piano: all 8 tones, silence, multiple keys, reset
$finish called at time : 1476 ns : File "C:/Users/yunji/ece2/ece2_2026/LAB5/FPGA/01_piano/sim/tb_piano.sv" Line 80
```

그림 12. 피아노 테스트벤치의 검사 결과.

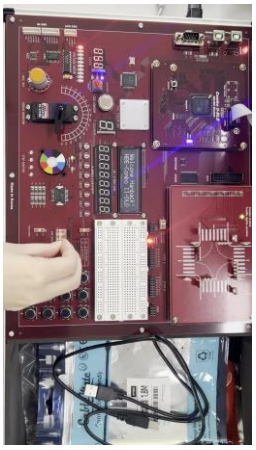


그림 13. 피아노 보드 시연.

FPGA 장치 xc7s75_0 의 Programmed 상태를 확인하고 피아노 동작을 시연하였다. 음높이는 출력의 반복 주파수로 결정되므로, 키별 분주값과 출력 반주기의 관계를 확인하는 것이 중요하다. 음계별 정량 평가는 각 출력 주파수의 계측이 필요하다.

피아노 RTL·TB·영상·Programmed 화면

4.2 신호등: 상태 진행 및 점멸

신호등은 방향과 단계 카운터를 이용해 차량의 적·황·녹색, 좌회전 및 보행자 출력을 제어한다. 시뮬레이션에서 방향 진행, 보행자 녹색 점멸, flicker 전환을 확인하였으며, 2200ns 에서 전체 주기·점멸·리셋 검사 PASS 를 확인하였다.

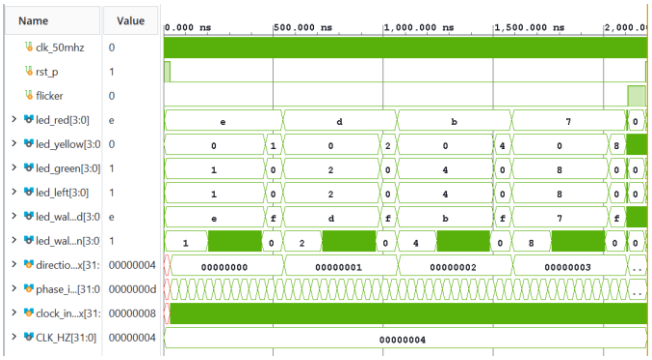


그림 14. 신호등 전체 상태 진행 시뮬레이션.

```
PASS traffic: full cycle, pedestrian blink, flicker, reset
$finish called at time : 2200 ns : File "C:/Users/yunji/ece2/ece2_2026/LAB5/FPGA/02_traffic/sim/tb_traffic.sv" Line 107
```

그림 15. 신호등 테스트벤치의 검사 결과.

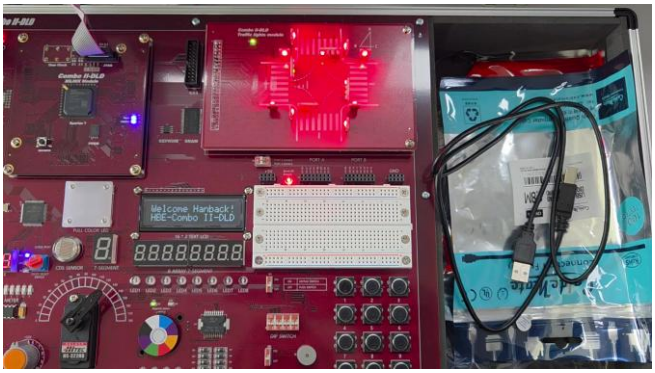


그림 16. 신호등 보드 시연.

녹색 출력 비트는 1→2→4→8 순서로 진행하였다. 방향 인덱스와 LED 비트의 대응을 일정하게 정해야 각 방향의 차량·보행자 신호가 일치한다. 상태별 출력은 적색·황색·녹색의 조합과 보행자 점멸 조건을 함께 확인한다.

신호등의 상태 순서와 실제 시간 길이는 각각 확인해야 한다. 상태 전이 파형으로 순서를 점검하고, 각 구간의 유지 시간은 보드 출력의 시간 간격을 측정해 평가한다. 축소된 시뮬레이션 시간은 실제 보드의 초 단위 동작 시간과 다르다.

신호등 RTL·TB·실제 시연

5. FPGA 추가 3 · 외부 ADC

5.1 직렬 변환 인터페이스

ADC 실습에서는 FPGA 로 외부 변환 장치를 제어하였다. FPGA 가 칩 선택, 직렬 클록, 채널 명령을 출력하고, 수신 비트를 시프트하여 sample 과 data_valid 를 생성한다. 변환 결과의 상위 비트는 LED 에 표시한다.[1,2]

```
PASS ADC: 8 frames, CH2->5->7, MOSI, 16-bit frames, tQUIET, dummy, sample, LED, valid pulse, reset
$finish called at time : 21271 ns : File "C:/Users/yunji/ece2/ece2_2026/LAB5/FPGA/03_adc/sim/tb_adc.sv" Line 161
```

그림 17. ADC 테스트벤치의 8 프레임 검사 결과.

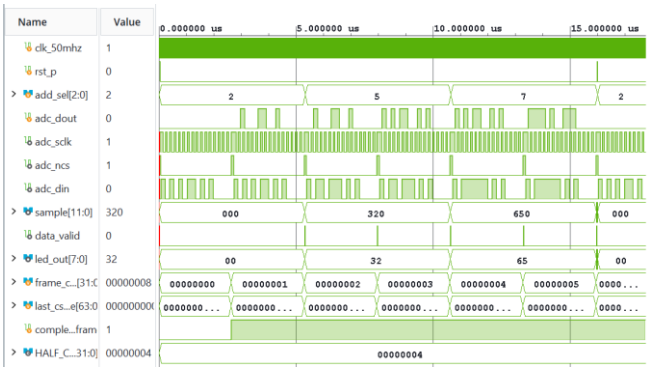


그림 18. CH2→5→7 변경, sample·data_valid·LED 파형.

ADC 테스트벤치에서는 CH2→5→7 전환, MOSI 명령, 16 비트 프레임, tQUIET, 초기 dummy 프레임, sample, LED, valid 펄스 및 리셋을 검사하였다. 파형의 sample=0x320·0x650 은 10 진수 800·1616 이며, led_out 은 각각 0x32·0x65 였다.

시뮬레이션 표본	10 진수	LED 표시
0x320	800	0x32
0x650	1616	0x65

표의 값은 ADC 디지털 모델의 시뮬레이션 표본값이다. 실제 입력 전압과의 관계는 사용 ADC 의 분해능, 기준 전압 및 전달 특성에 따라 결정된다.

5.2 보드 동작 및 검증 항목



그림 19. 외부 ADC 의 보드 및 입력 연결.



그림 20. 외부 ADC 보드 시연.

보드 시연에서는 외부 ADC 의 입력 구성과 LED 출력을 확인하였다. 전압 환산 정확도와 선형성을 평가하려면 입력 전압과 디지털 코드를 동시에 측정하고, 기준 전압 Vref 및 장치의 전달 특성과 비교해야 한다.

직렬 수신은 프레임 길이, 샘플링 에지, 채널 명령과 변환 결과의 대응이 핵심이다. 채널 변경 시에는 해당 장치의 변환 지연을 고려해야 하며, data_valid 가 한 클록 동안 발생하는지와 LED 가 수신 표본의 상위 비트를 나타내는지를 함께 확인한다.

ADC 소스 및 시연 결과

6. 고찰 및 결론

6.1 비교와 문제 해결

항목	확인된 결과	남은 검증
과제 3	보드 시연·ISR 동작	1/2/4/8 초 동시 계측
과제 4	CCR 순환·듀티 3 종 실측	반복 측정·주파수 오차 원인
피아노	8 키 TB PASS·보드 시연	음계별 주파수 계측
신호등	상태·점멸 TB·보드 시연	상태별 시간·출력 매핑
ADC	프레임 TB·보드 시연	모델/Vref·전압-코드 표

PWM 의 CCR 은 한 주기 안에서 출력이 전환되는 비교값이다. 따라서 CCR=250·500·750 의 변화는 주파수 변경이 아니라 25·50·75% 듀티 변경을 의미한다. Watch 는 변수 갱신 순서를 확인하는 데 사용하고, 정상 실행 파형으로 주기와 듀티를 측정하였다.

인터럽트 방식은 버튼 입력과 주기 동작의 역할을 분리할 수 있다. 공유 변수에는 volatile 을 적용하되 복합 갱신 중 선점을 고려해야 한다. FPGA 에서는 상태 전이와 출력 매핑, ADC 의 프레임 경계 및 유효 신호가 주요 점검 항목이었다.

6.2 결론

PWM 의 목표 듀티 25·50·75%에 대해 실측값은 24.79·50.34·74.98%였으며 최대 편차는 0.34%p 였다. CCR 순환과 출력 듀티의 관계를 확인하였다. LED 제어에서는 버튼과 타이머 ISR 의 역할을 이해하고, FPGA 실습에서는 음 출력·신호등 상태 전이·ADC 직렬 수신의 동작을 확인하였다.

6.3 소스 코드 및 제출 버전

https://github.com/YunchanJu/lab4_stm32_0

제출 태그: **stm32-1-results-20261019-v2**
소스·증거 기준 커밋:
cf68ea5bc775d231aafca292da9399f31893b6d0

소스 코드와 실험 영상은 아래 저장소의 각 실험 폴더에서 확인할 수 있다. 제출 버전은 태그와 커밋 해시로 표시하였다.

과제 3 영상 · 과제 4 영상

피아노 영상 · 신호등 영상 · ADC 영상

실험별 소스 코드 · 시뮬레이션 결과 · 측정 기록

참고문헌

[1] thechef030, 「STM32_1 · 인터럽트와 PWM」, 4 주 실습 매뉴얼. 2026-10-26 확인. 버튼·타이머·PWM 요구사항을 요약·참고하였다.
교안 STM32_1 원문

[2] thechef030, 「FPGA 모듈별 Verilog 소스와 테스트벤치」, FPGA_additional.md. 피아노·신호등·ADC 구조와 테스트 항목 참고. 2026-10-26 확인.
FPGA 교안 원문

[3] STMicroelectronics, *STM32G4 Nucleo-64 boards (MB1367)*, User manual **UM2505**. 보드 개요와 연결 구성 참고.
ST 공식 사용자 매뉴얼

그림 1~20 은 과제 3·4 와 FPGA 피아노·신호등·ADC 의 회로 구성, 동작 및 파형을 나타낸다. PWM 표는 오실로스코프 측정값이며, ADC 표는 테스트벤치의 디지털 표본값이다.